gc_content.py 코드 해설

> lab11 · 생물정보학실험
> 대상 파일: `lab11/src/gc_content.py`
> 실행 방법과 결과 해석은 `gc_content_usage.md`를 참고하세요.

1. 코드 전문

```
import sys

def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()

    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip().upper()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))

    return records


def gc_content(seq):
    gc = seq.count("G") + seq.count("C")
    atgc = sum(seq.count(base) for base in "ATGC")
    if atgc == 0:
        return 0
    return gc / atgc * 100


def mean_quality(qual):
    scores = [ord(ch) - 33 for ch in qual]
    return sum(scores) / len(scores)


reads = read_fastq(sys.argv[1])

high_gc = []
low_gc = []

for name, seq, qual in reads:
```

```
    gc = gc_content(seq)
    q = mean_quality(qual)

    if gc >= 50:
        high_gc.append(q)
    else:
        low_gc.append(q)


print("전체 read 개수:", len(reads))

print("GC 비율 높은 read 수:", len(high_gc))
print("GC 비율 낮은 read 수:", len(low_gc))

if high_gc:
    print("GC 높은 그룹 평균 품질:",
          sum(high_gc) / len(high_gc))

if low_gc:
    print("GC 낮은 그룹 평균 품질:",
          sum(low_gc) / len(low_gc))

if high_gc and low_gc:
    diff = (sum(high_gc) / len(high_gc)
            - sum(low_gc) / len(low_gc))
    print("평균 품질 차이 (높음 - 낮음):", diff)
```

코드는 크게 여섯 부분으로 나뉩니다.

부분	하는 일
① 준비	`sys` 모듈 불러오기
② `read_fastq()` 함수	FASTQ 파일을 읽어 (이름, 서열, 품질) 목록 만들기
③ `gc_content()` 함수	서열 하나의 GC 비율(%) 계산
④ `mean_quality()` 함수	품질 문자열 하나의 평균 Phred 점수 계산
⑤ 그룹 나누기	read마다 GC 50 % 기준으로 높은/낮은 그룹에 품질 점수 넣기
⑥ 출력	read 수, 그룹별 평균 품질, 두 그룹의 차이 출력

2. ① 준비

```
import sys
```

코드	해설
`import sys`	명령줄에서 입력한 값(파일 이름)을 읽기 위해 `sys` 모듈을 불러온다. 나중에 `sys.argv[1]`로 파일 이름을 꺼낸다.

3. ② `read_fastq()` 함수

```python
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()

    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip().upper()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))

    return records
```

코드	해설
`def read_fastq(filename):`	파일 이름을 받아 FASTQ를 읽는 함수를 만든다.
`records = []`	결과를 담을 빈 리스트를 만든다.
`with open(filename) as f:`	파일을 연다. `with`를 쓰면 다 쓴 뒤 파일이 자동으로 닫힌다.
`lines = f.readlines()`	파일의 모든 줄을 한꺼번에 읽어 리스트로 만든다. `lines[0]`이 첫 줄.
`for i in range(0, len(lines), 4):`	i를 0, 4, 8, …로 4씩 늘린다. FASTQ는 **4줄이 read 하나**이므로 i는 항상 각 read의 첫 줄 번호다.
`name = lines[i].rstrip()[1:]`	1번째 줄(`@이름`). `rstrip()`으로 줄바꿈을 지우고 `[1:]`로 맨 앞 `@`를 뗀다.
`seq = lines[i + 1].rstrip().upper()`	2번째 줄: 염기 서열. **`.upper()`로 대문자로 바꾼다.** 소문자 `g`, `c`가 있어도 GC 계산에서 빠지지 않게 하기 위해서다.
`qual = lines[i + 3].rstrip()`	4번째 줄: 품질 문자열. 3번째 줄(`+`)은 필요 없어서 건너뛴다. 품질 문자는 대소문자가 서로 다른 점수이므로 **바꾸면 안 된다.**
`records.append((name, seq, qual))`	세 값을 튜플 하나로 묶어 리스트에 추가한다.
`return records`	완성된 리스트를 돌려준다.

> `qual_by_position.py`의 `read_fastq()`와 거의 같지만, **서열에 `.upper()`가 붙은 점**이 다릅니다. 이 프로그램은 서열(GC)을 실제로 쓰기 때문입니다.

예를 들어 실습 파일의 첫 read는 아래 튜플이 됩니다.

```
("SRR1553607.1 1 length=101",
 "GTTAGCGTTGTTGATCGC...",
 "BB@FFFFFHHHHHJJJJJ...")
```

---

4. ③ `gc_content()` 함수

```python
def gc_content(seq):
    gc = seq.count("G") + seq.count("C")
    atgc = sum(seq.count(base) for base in "ATGC")
    if atgc == 0:
        return 0
    return gc / atgc * 100
```

코드	해설
`def gc_content(seq):`	서열 문자열 하나를 받아 GC 비율을 돌려주는 함수.
`gc = seq.count("G") + seq.count("C")`	`count()`는 문자열 안에 해당 글자가 몇 개 있는지 센다. G 개수와 C 개수를 더한다.
`atgc = sum(seq.count(base) for base in "ATGC")`	`"ATGC"`를 한 글자씩 `base`로 꺼내 각각 개수를 센 뒤 `sum()`으로 모두 더한다. 즉 A + T + G + C 개수.
`if atgc == 0:`	A/T/G/C가 하나도 없는 경우(예: 서열이 전부 `N`)를 확인한다.
`return 0`	이때 그대로 나누면 `ZeroDivisionError`가 나므로 0을 돌려준다.
`return gc / atgc * 100`	비율을 계산해 **퍼센트(%)** 로 돌려준다.

`sum(seq.count(base) for base in "ATGC")`는 아래 코드를 한 줄로 줄인 것입니다.

```python
atgc = 0
for base in "ATGC":
    atgc += seq.count(base)
```

분모가 `len(seq)`가 아닌 이유: 시퀀서가 판단하지 못한 염기는 `N`으로 표시됩니다. `len(seq)`로 나누면 `N`까지 분모에 들어가 GC 비율이 실제보다 낮게 나옵니다. A·T·G·C 개수만 분모로 쓰면 `N`은 계산에서 빠집니다.

```
seq = "GGCCNN"
len(seq) 로 나누면 : 4 / 6 × 100 = 66.7 %
atgc 로 나누면     : 4 / 4 × 100 = 100.0 %   ← 이 코드의 방식
```

---

5. ④ `mean_quality()` 함수

```python
def mean_quality(qual):
    scores = [ord(ch) - 33 for ch in qual]
    return sum(scores) / len(scores)
```

코드	해설
`def mean_quality(qual):`	품질 문자열 하나를 받아 평균 품질 점수를 돌려주는 함수.
`scores = [ord(ch) - 33 for ch in qual]`	품질 문자열을 한 글자씩 `ch`로 꺼내 점수로 바꾼 리스트를 만든다(**리스트 컴프리헨션**). `ord()`는 문자의 ASCII 번호를 주고, Phred+33 규칙에 따라 33을 뺀다.
`return sum(scores) / len(scores)`	점수의 합 ÷ 점수 개수 = 이 read의 **평균 품질 점수**.

리스트 컴프리헨션은 아래 코드와 같은 뜻입니다.

```python
scores = []
for ch in qual:
    scores.append(ord(ch) - 33)
```

변환 예시:

```
qual   = "BB@F"
ord()  =  66  66  64  70
-33    =  33  33  31  37
평균   = (33 + 33 + 31 + 37) / 4 = 33.5
```

> 품질 줄이 비어 있으면 `len(scores)`가 0이 되어 `ZeroDivisionError`가 납니다. `gc_content()`와 달리 이 경우를 따로 막는 코드는 없습니다.

---

6. ⑤ 그룹 나누기

```python
reads = read_fastq(sys.argv[1])
```

```
high_gc = []
low_gc = []
```

코드	해설
`reads = read_fastq(sys.argv[1])`	명령줄의 첫 번째 값(파일 이름)으로 함수를 불러 모든 read를 `reads`에 저장한다. 파일 이름을 안 주면 여기서 `IndexError`가 난다.
`high_gc = []`	GC 50 % 이상인 read들의 **평균 품질 점수**를 담을 리스트.
`low_gc = []`	GC 50 % 미만인 read들의 **평균 품질 점수**를 담을 리스트.

두 리스트에는 서열이 아니라 **숫자(평균 품질)** 만 들어갑니다. 나중에 평균을 내는 데는 그것만 있으면 되기 때문입니다.

```
for name, seq, qual in reads:
    gc = gc_content(seq)
    q = mean_quality(qual)

    if gc >= 50:
        high_gc.append(q)
    else:
        low_gc.append(q)
```

코드	해설
`for name, seq, qual in reads:`	read를 하나씩 꺼내고, 튜플의 세 값을 `name`, `seq`, `qual`에 나눠 담는다. `name`은 쓰이지 않는다.
`gc = gc_content(seq)`	이 read의 GC 비율(%)을 계산한다.
`q = mean_quality(qual)`	이 read의 평균 품질 점수를 계산한다.
`if gc >= 50:`	GC가 **50 이상**(50.0 포함)인지 확인한다.
`high_gc.append(q)`	그렇다면 품질 점수를 높은 그룹에 넣는다.
`else:` / `low_gc.append(q)`	아니면(50 미만) 낮은 그룹에 넣는다.

예시 흐름:

```
read 1 : GC 42.6 %, Q 37.4  →  low_gc  = [37.4]
read 2 : GC 51.5 %, Q 34.0  →  high_gc = [34.0]
read 3 : GC 63.4 %, Q 30.1  →  high_gc = [34.0, 30.1]
```

---

7. ⑥ 출력

```
print("전체 read 개수:", len(reads))

print("GC 비율 높은 read 수:", len(high_gc))
print("GC 비율 낮은 read 수:", len(low_gc))
```

코드	해설
`len(reads)`	전체 read 수.
`len(high_gc)`, `len(low_gc)`	각 그룹에 들어간 read 수. 두 값을 더하면 전체 read 수와 같다.
`print("...", 값)`	쉼표로 이어 쓰면 사이에 공백 하나를 넣어 함께 출력한다.

```
if high_gc:
    print("GC 높은 그룹 평균 품질:",
          sum(high_gc) / len(high_gc))

if low_gc:
    print("GC 낮은 그룹 평균 품질:",
          sum(low_gc) / len(low_gc))
```

코드	해설
`if high_gc:`	리스트가 비어 있으면 `False`, 하나라도 있으면 `True`로 취급된다. 빈 그룹이면 출력을 건너뛰어 **0으로 나누는 오류를 막는다.**
`sum(high_gc) / len(high_gc)`	그룹 안 read들의 평균 품질을 다시 평균 낸 값(**평균의 평균**). read 길이와 상관없이 read마다 한 표씩 반영된다.
괄호 안 줄바꿈	`print(` 괄호가 아직 닫히지 않았으므로 다음 줄로 넘겨 써도 한 문장으로 인식된다.

```
if high_gc and low_gc:
    diff = (sum(high_gc) / len(high_gc)
            - sum(low_gc) / len(low_gc))
    print("평균 품질 차이 (높음 - 낮음):", diff)
```

코드	해설
`if high_gc and low_gc:`	두 그룹이 **모두** 비어 있지 않을 때만 차이를 계산한다.
`diff = (... - ...)`	높은 그룹 평균 − 낮은 그룹 평균. 바깥 괄호 덕분에 두 줄로 나눠 쓸 수 있다.
`print(..., diff)`	차이를 출력한다. 음수면 GC 높은 그룹의 품질이 더 낮다는 뜻이다.

출력 예시 (`data/SRR1553607_1.fastq`):

```
전체 read 개수: 203445
GC 비율 높은 read 수: 165738
GC 비율 낮은 read 수: 37707
GC 높은 그룹 평균 품질: 33.22470423018749
GC 낮은 그룹 평균 품질: 34.43877059358424
평균 품질 차이 (높음 - 낮음): -1.2140663633967463
```

평균 값은 반올림 없이 그대로 출력되므로 소수점 아래 자리가 깁니다.

---

8. 한눈에 보는 흐름

```
FASTQ 파일
    │  read_fastq()  : 4줄씩 끊어 (이름, 서열(대문자), 품질) 목록 만들기
    ▼
reads 리스트
    │  read마다
    │    - gc_content(seq)    → GC 비율 (%)
    │    - mean_quality(qual) → 평균 품질 점수
    │    - GC ≥ 50 ? → high_gc 에 추가
    │                아니면 → low_gc 에 추가
    ▼
high_gc / low_gc 리스트 (평균 품질 점수들)
    │  sum ÷ len
    ▼
그룹별 read 수, 평균 품질, 차이(높음 − 낮음) 출력
```

---

9. 알아 두면 좋은 점

- **함수로 나눠 놓은 이유**: GC 계산과 품질 계산을 각각 함수로 만들어 두면, 다른 프로그램에서도 그대로 가져다 쓸 수 있고 따로 시험해 보기도 쉽습니다.
- **`name`은 쓰이지 않습니다.** `read_fastq()`를 다른 lab11 프로그램과 같은 모양으로 맞춰 두었기 때문에 함께 읽어 오는 것입니다.
- **0으로 나누기 방어가 부분적입니다.** `gc_content()`와 출력 부분은 0으로 나누는 경우를 막지만, `mean_quality()`는 막지 않습니다.
- **파일 전체를 메모리에 올립니다.** `readlines()`가 모든 줄을 한 번에 읽으므로, 아주 큰 FASTQ 파일에서는 메모리가 부족할 수 있습니다.
- **FASTQ 형식이 정확해야 합니다.** 빈 줄이 끼어 있거나 줄 수가 4의 배수가 아니면 `IndexError`가 나거나 엉뚱한 줄을 품질로 읽게 됩니다.
- **기준값 50 은 코드에 직접 적혀 있습니다.** 다른 기준(예: 60 %)으로 나눠 보려면 `if gc >= 50:`의 숫자를 바꿔야 합니다.